Design Tinder

SystemsExpert design walkthrough

Clarifying questions

- Profiles, swiping, matches, chat - which are in scope?
- How are recommendations generated?

Functional requirements

- Create/update profile and preferences
- Get a deck of candidate profiles
- Swipe left/right; mutual right swipe creates a match
- List matches; message matches

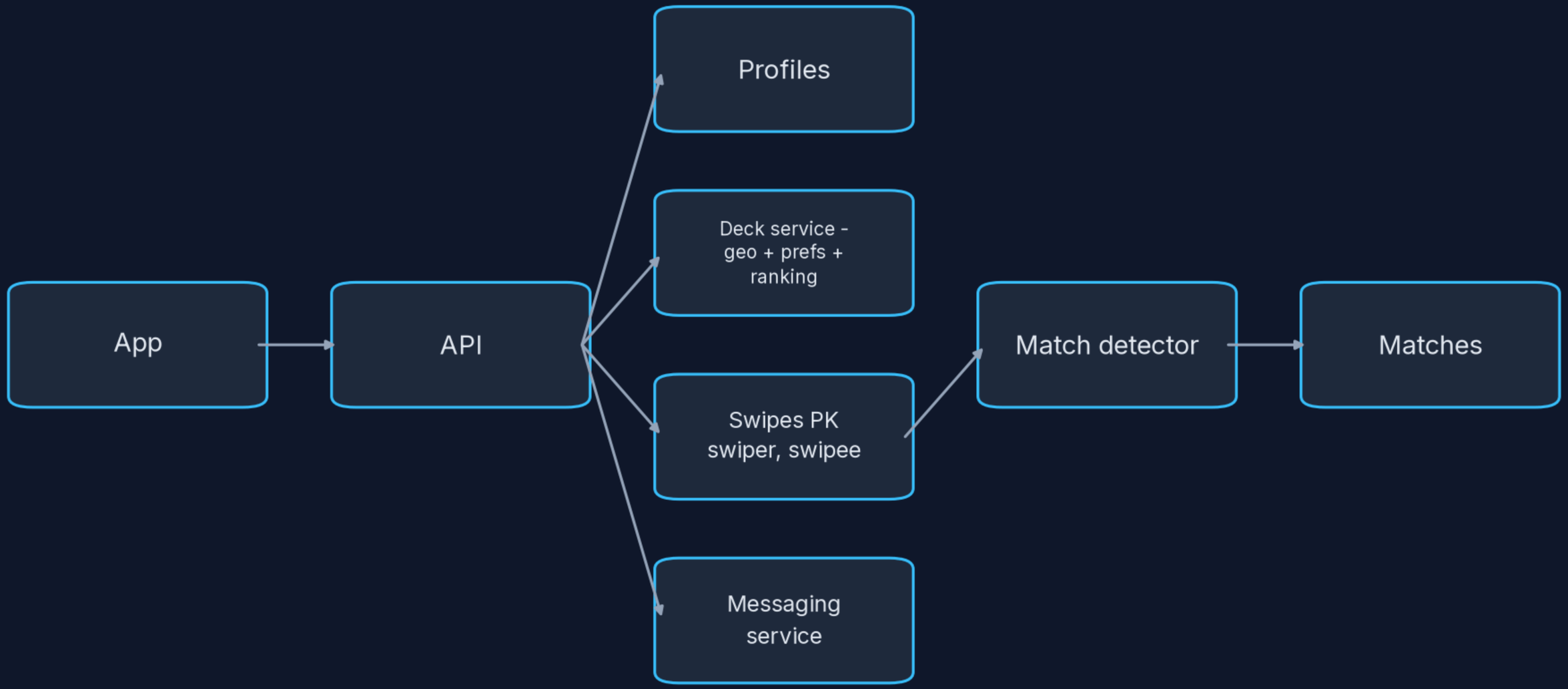
Non-functional requirements

- Deck loads quickly; swipes never lost
- Match created exactly once

Capacity estimates

- ~50M DAU x ~100 swipes/day ~ 60k swipes/s peak

High-level architecture



Data model

- Profile(user_id, name, photos, bio, geo, prefs)
- Swipe(swiper_id, swipee_id, decision, ts)
- Match(id, user_a, user_b, created_at) unique(min,max)

API

```
GET  /v1/profile, PUT /v1/profile
GET  /v1/deck?limit=50              -> profiles[]
POST /v1/swipes {profileId, decision: LIKE|PASS} -> {matched: bool, matchId?}
GET  /v1/matches?cursor
POST /v1/matches/{id}/messages {text}
GET  /v1/matches/{id}/messages?cursor
```

Match detection

- On LIKE, check whether the reverse LIKE exists; insert the match with a unique constraint on the ordered pair so concurrent likes create one match.

Deck

- Precompute decks in the background per user; exclude already-swiped profiles with a bloom filter.

Trade-offs & alternatives

- Synchronous match check gives instant 'It's a match!' at slightly higher write latency
- Precomputed decks may be slightly stale

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: App, API, Profiles, Deck service - geo + prefs + ranking, Swipes PK swiper, swipee
- Know your trade-offs and failure modes